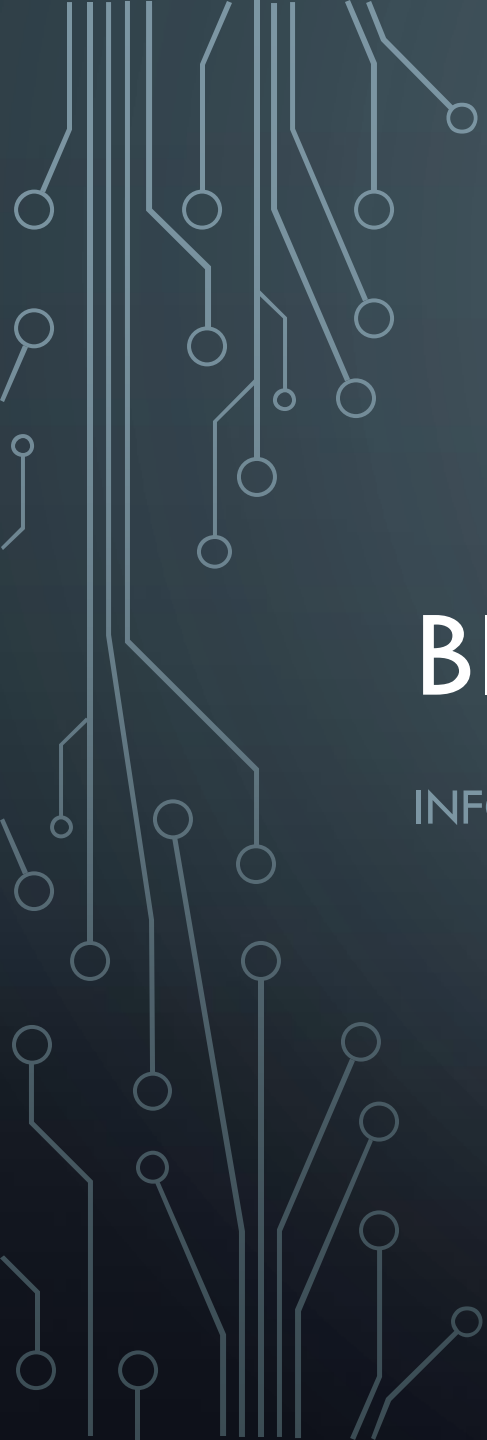


An abstract graphic on the left side of the slide, consisting of a network of white lines and circles on a dark blue background. The lines are vertical and horizontal, with some diagonal branches, and the circles are of varying sizes, resembling a circuit board or a data network.

BIG DATA

INFORMATION RETRIEVAL

LEARNING OBJECTIVES

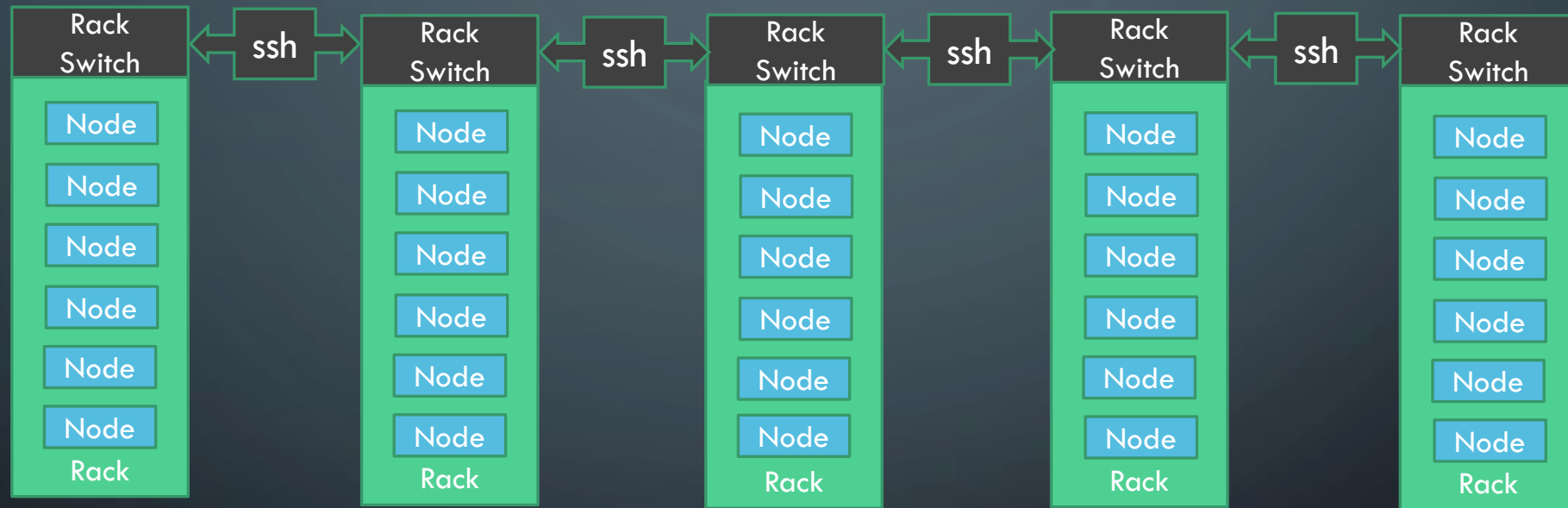
- At the end of this presentation, you should be able to
 - Explain how the Hadoop infrastructure facilitates Big Data processing;
 - Explain the role of Pig in making MapReduce processing more accessible;
 - Explain the role of Hive as an analytical tool for structured data;
 - Discuss how these technologies change the way we traditionally processed data.

MAPREDUCE IS DIFFICULT

- MR is for experienced programmers
- Code has to be compiled and deployed
- Although you can write it in many languages

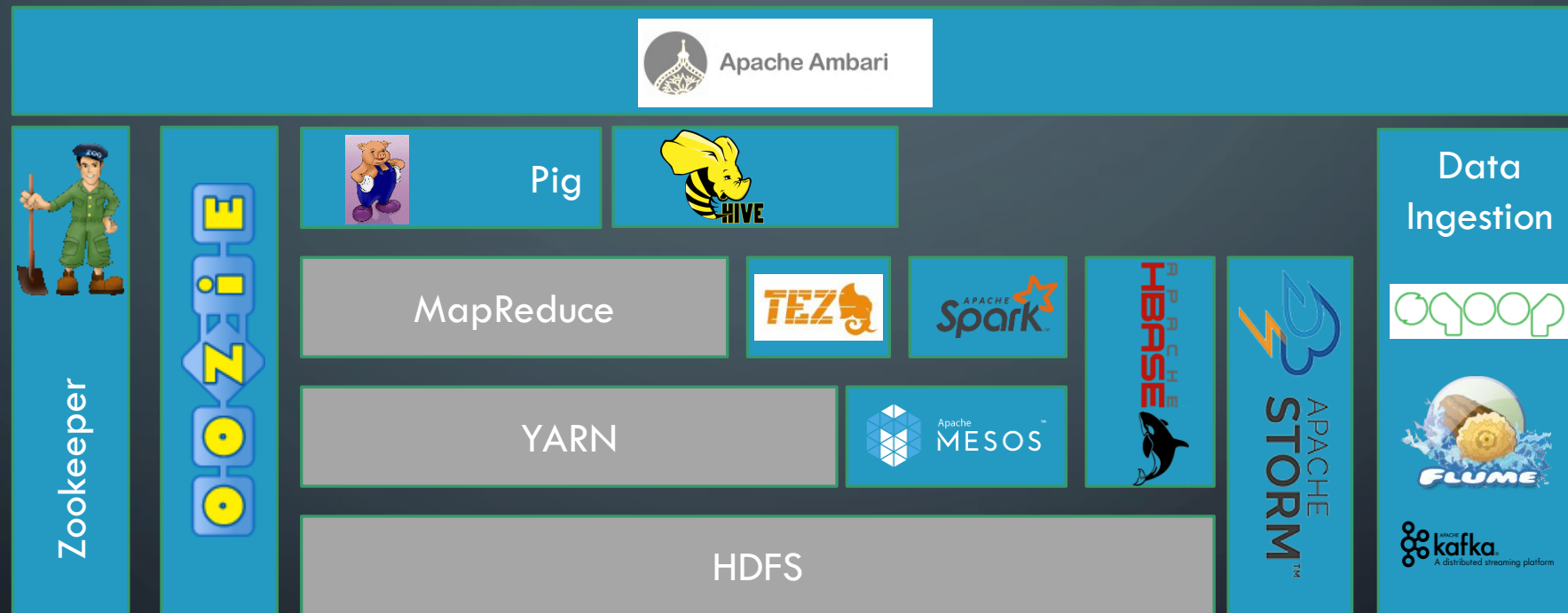


HADOOP CLUSTERS ARE GREAT



but MapReduce is complicated!

HADOOP 'ECOSYSTEM'



AND ALSO...



Apache Zeppelin

PIG

Hadoop Tool for unstructured data



Apache Pig

PIG - MAPREDUCE



Apache Pig

PIG

- High-level data processing
- Commands for joins, filters, groups
- Nested data types like maps, bags, tuples
- Cannot replace MR in all cases

MAPREDUCE

- Low-level programmatic processing
- Detailed coding is necessary
- Explicit data structures like lists, tuples, maps
- Complex analysis can be coded

PIG RUNNING MODES



Apache Pig

MAPREDUCE MODE

- Runs on Hadoop
- Uses the MR parallelisation
- Works off the HDFS file system
 - Reads of HDFS
 - Writes to HDFS

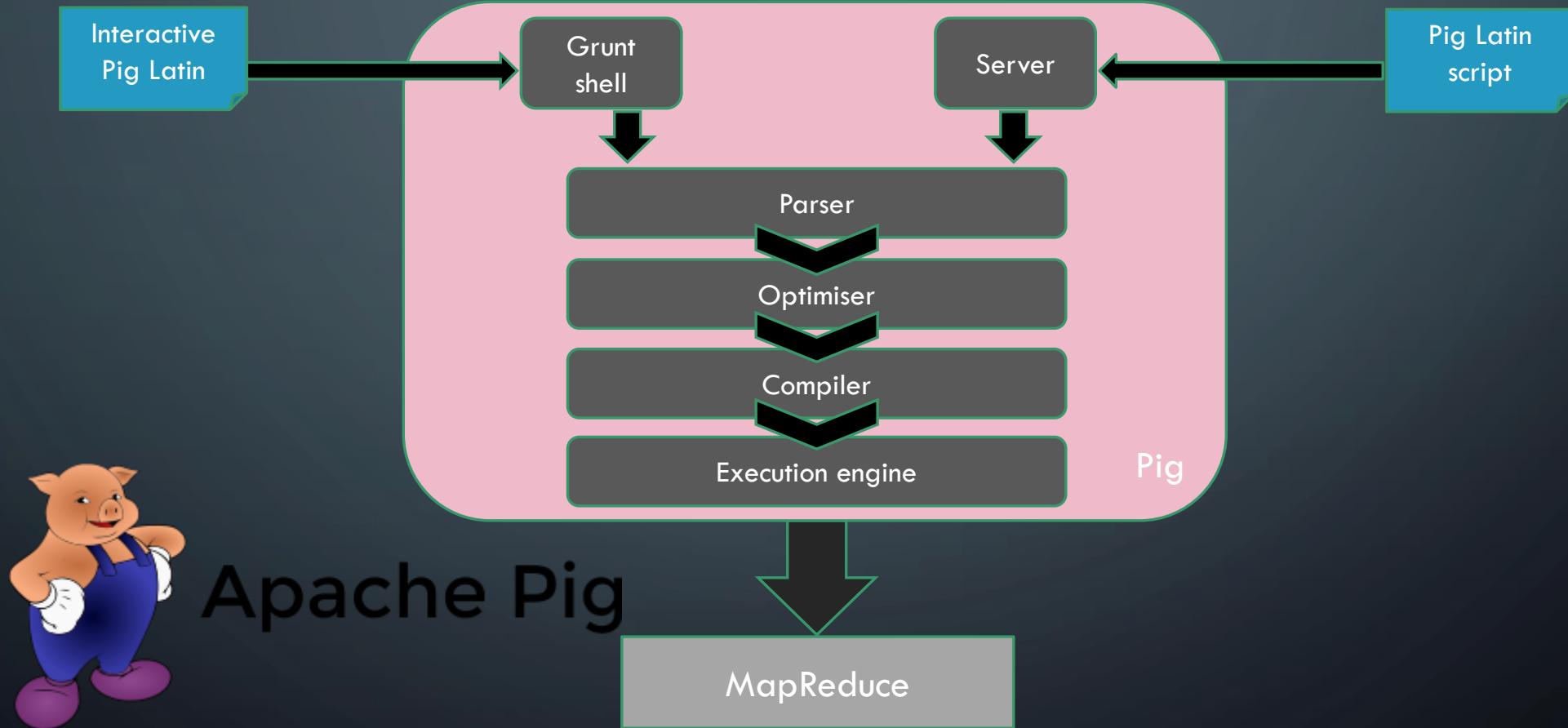
Command: `pig`

LOCAL MODE

- Runs on a single computer
- Runs a query sequentially
- Works from the local file system
 - Reads one file
 - Writes one file (one reducer)

Command: `pig -x local`

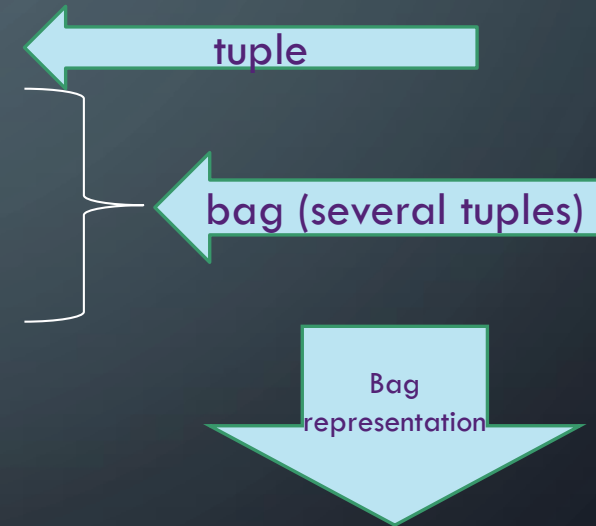
PIG ARCHITECTURE



PIG DATA STRUCTURES – TUPLE AND BAG

ID	Name	Street	Phone
1	Bob	Lithcombe	999345
2	Wang	Croyden	890999
3	Srikanth	Kew	433534
4	Veronica	Hawthorn	345490

Tuple is
pronounced
'tyoople'
!



{(2, Wang, Croyden, 890999), (3, Srikanth, Kew, 433534), (4, Veronica, Hawthorn, 345490)}

PIG DATA STRUCTURES – MAP



Id # 1,
Name # Bob,
Address # Lithcombe,
Phone # 934582

Id # 2,
Name # Wang,
Address # Croyden,
Phone # 4335424



key # value

Atom in Pig Latin means 'primitive type' like int, char, float, double

EXAMPLE FILE

SalesJan2009.csv

Transaction_date	Product	Price	Payment	Name	City	State
7/8/20 22:00	Product1	1200	Mastercard	Carolina	Basildon	England

PIG LATIN



Apache Pig

```
rawsales = LOAD 'xxx.csv' USING PigStorage(',') AS (Transaction_date:chararray, Product:chararray, Price:int, ....);
```

```
salesdata = FILTER rawsales BY Price IS NOT NULL;
```

```
salesandcountry = FOREACH salesdata GENERATE (chararray) Country AS country, (int) Price AS price;
```

```
countrygroups = GROUP salesandcountry BY country;
```

```
salespercountry = FOREACH countrygroups GENERATE group as country, SUM(salesandcountry.price) AS totalsales;
```

```
sortedsales = ORDER salespercountry BY totalsales desc;
```

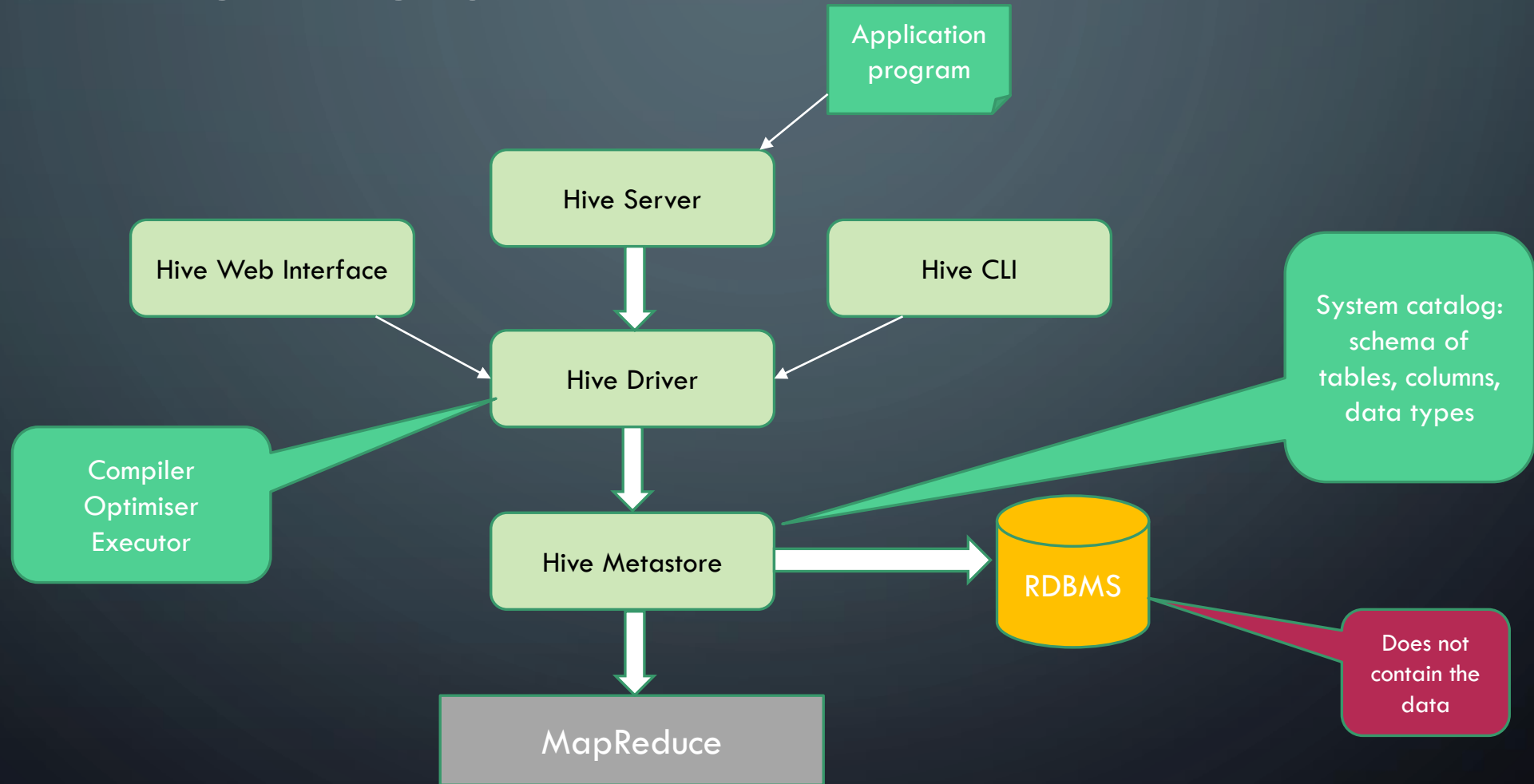
```
STORE sortedsales INTO '/pigdata/SalesByCountry' USING PigStorage(' ');
```

HIVE

Hadoop Tool for structured data



HIVE ARCHITECTURE



HIVEQL (HQL)

- Includes basic SQL statements:

- FROM clause & subqueries `SELECT name FROM (SELECT * FROM employee WHERE salary > 5000);`
- JOIN `SELECT name, role FROM employee e JOIN role r ON e.role_id = r.id;`
- multi-table INSERT `Details later – very useful for building an RDB from a .csv file`
- multi-grouping `SELECT SUM(total), year, month, location .... GROUP BY year, month, location;`

- AND ALSO

- TRANSFORM for pluggable scripts `Details later – very useful for changing things on the fly`

MULTI-TABLE INSERT - NORMALISING

Sales.csv

Time	Product	Salesrep_id	Salesrep_name	Salesrep_phone	Cust_id	Cust_name
7/8/20 22:00	Trailer	13001	Pete	0993495	200021	Song



Salesrep

Salesrep_id	Salesrep_name	Salesrep_phone
13001	Pete	0993495

Customer

Cust_id	Cust_name
200021	Song

Sale

Time	Product	Salesrep_id	Cust_id
7/8/20 22:00	Trailer	13001	200021

MULTI-TABLE INSERT - HQL

FROM Sales.csv s

INSERT OVERWRITE TABLE Sale SELECT s.Time, s.Product, s.Salesrep_id, s.Cust_id

INSERT OVERWRITE TABLE Customer SELECT s.Cust_id, s. Cust_name

INSERT OVERWRITE TABLE Salesrep SELECT s.Salesrep_id, s.Salesrep_name, s.Salesrep_phone;

IS HIVE A RELATIONAL DATABASE?

NO!

HIVE

- Schema on read
 - Storage in files on HDFS
 - No OLTP (it's a data warehouse!)
- It's for analysis



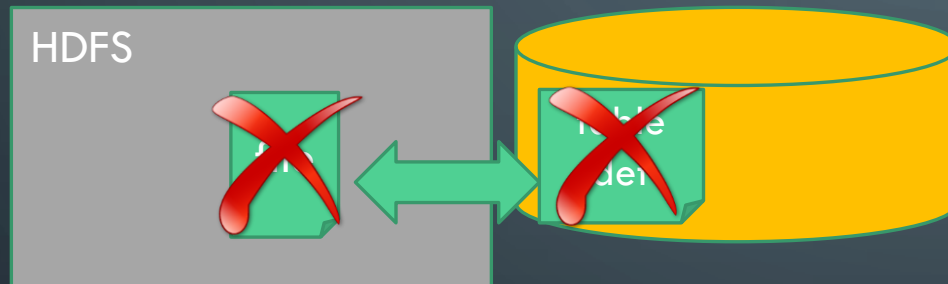
RDBMS

- Schema on write
 - Storage in internal custom file system
 - OLTP
- It's for storage



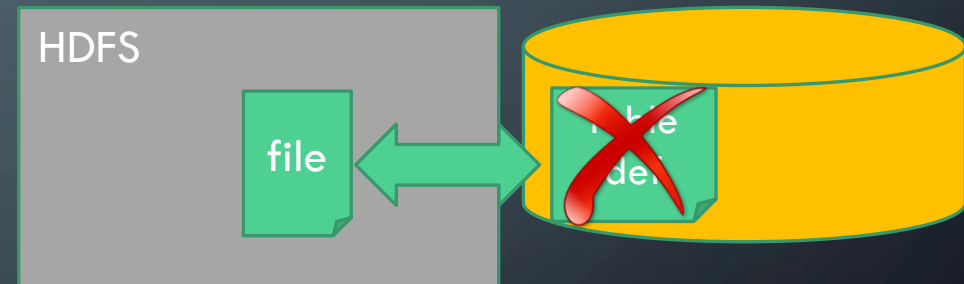
MANAGED AND EXTERNAL TABLES

MANAGED



```
CREATE TABLE <tablename>  
(col1 datatype, col2 datatype)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'  
STORED as textfile;
```

EXTERNAL



```
CREATE EXTERNAL TABLE <tablename>  
(col1 datatype, col2 datatype)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'  
STORED as textfile;
```

HIVE LATENCY ISSUES

```
SELECT * FROM sale
WHERE time BETWEEN
2020-08-01 AND 2020-08-31;
```

/usr/datastore/

part_00001

part_00002

part_00003

part_00004

Part files
can still be
very large

Part files
containing sales

sale

Time	Product	Salesrep_id	Cust_id
7/8/20 22:00	Trailer	13001	200021

HIVE PARTITIONS

/usr/datastore/time=2020-08-01

part_00001

part_00002

part_00003

part_00004

/usr/datastore/time=2020-08-02

part_00001

part_00002

part_00003

part_00004

Store by
one column

sale

Time	Product	Salesrep_id	Cust_id
7/8/20 22:00	Trailer	13001	200021

HIVE BUCKETS

/usr/datastore/time=2020-08-02

Secondary
column

part_00001

Salesrep 13001, 13002, 13003

part_00002

Salesrep 13004, 13005, 13006

part_00003

Salesrep 13007, 13008, 13009

part_00004

Salesrep 13010, 13011, 13012

part_00005

Salesrep 13013, 13014, 13015

CREATING PARTITIONS AND BUCKETS

CREATE TABLE sales

PARTITION BY (time)

CLUSTERED BY (salesrep_id) INTO 5 BUCKETS

PIG - HIVE

PIG



- Procedural language
 - For programmers
- Executed on the client
- Inspired by SQL

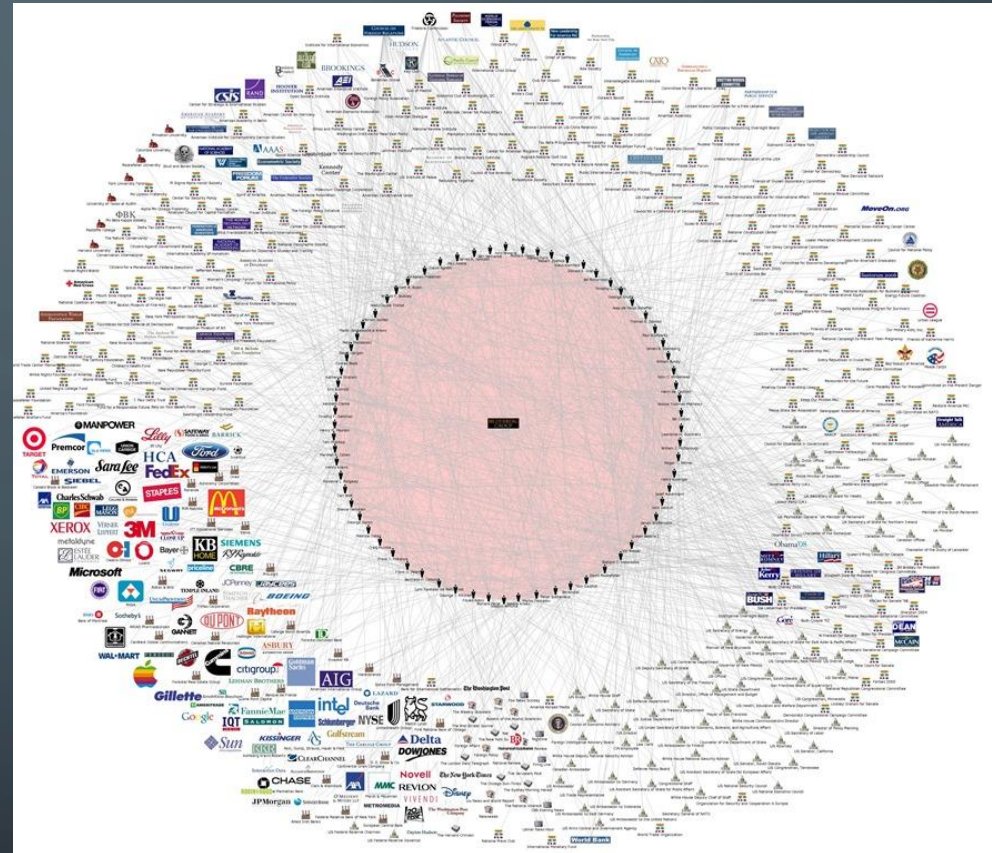
HIVE



- Declarative language
 - For data scientists
- Executed on the server
- Based on SQL

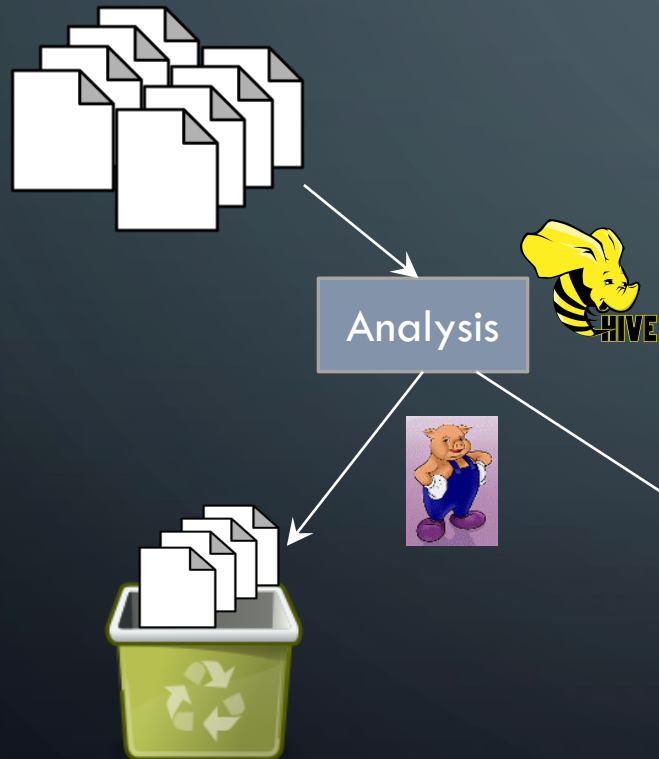
THE BIG PICTURE

Data processing in the big data age

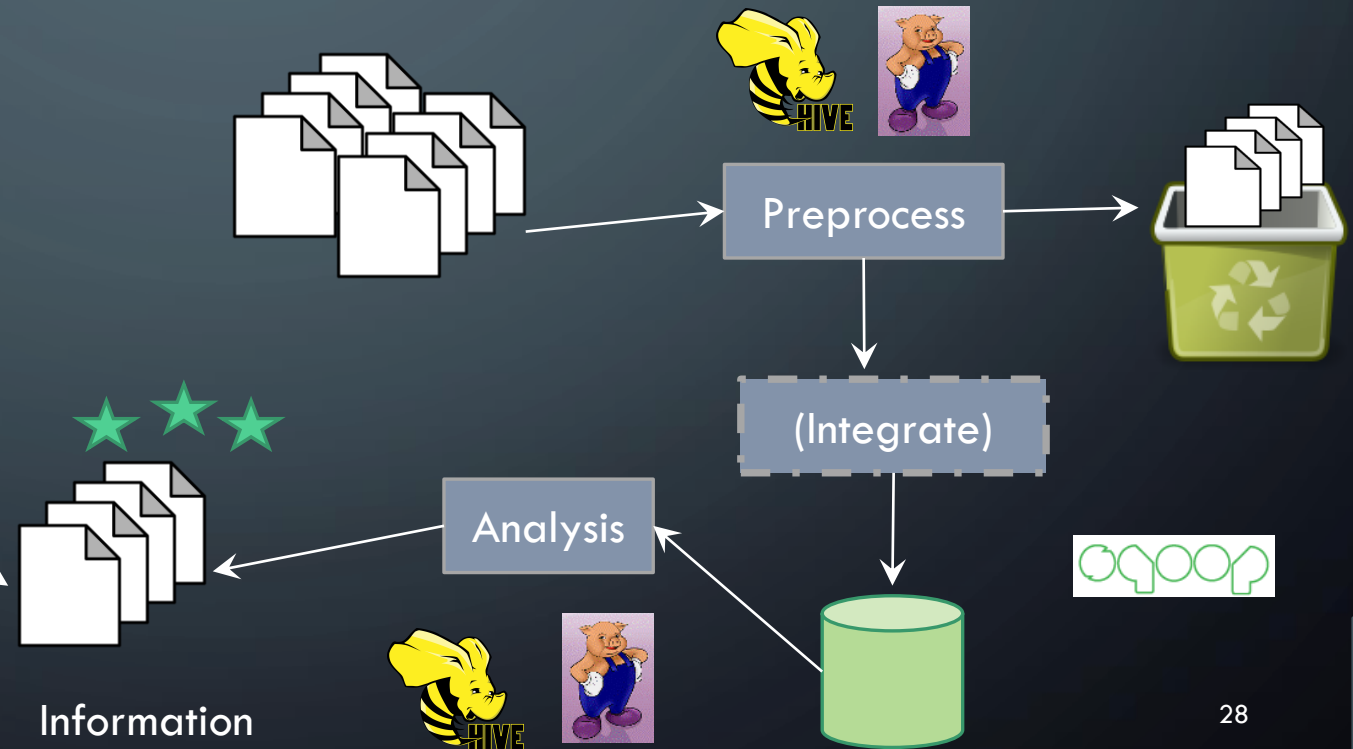


HOW DO WE DEAL WITH BIG DATA?

- Access only once



- Access several times



SUMMARY

- Hadoop and Apache tools provide the means to process big data sets in a versatile way.
- Pig and Hive are information retrieval tools that are easy to use.
- Parallelisation and distribution are the underlying principle of all tools.
- Information retrieval can be part of the preprocessing or transformation phase.
 - Big data processing is not rigid ETL.